

1) CPU scheduling in a Real time Ticket Booking System:

1) Most suitable scheduling Algorithm

⇒ Priority Scheduling

2) Justification

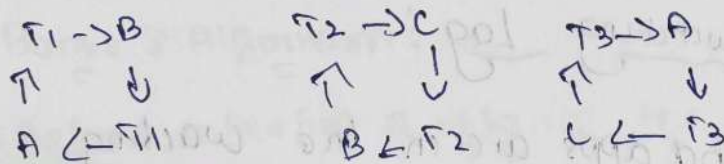
- Payment transaction (P1) are the most important and most
- Seat availability checks (P2) are executed after Payment request
- Ticket confirmation and printing (P3)

3) P1	High	5ms
P2	Medium	10ms
P3	Low	15ms

4) one Drawback:

⇒ Starvation: Low priority processes (such as Ticket confirmation) may wait for a long time.

P1 → P2 → P3

2) Deadlock in a Banking Transaction Server:

T1 holds A → request B

T2 holds B → request C

T3 holds C → request A

2) Has a Deadlock occurred:-

Yes, a deadlock has occurred.

• T_1 waits for B, which is held by T_2 .

• T_2 waits for C, which is held by T_3 .

• T_3 waits for A, which is held by T_1 .

3) Recommended technique

Deadlock Prevention

- Enforce a fixed order for acquiring resource
- Every transaction must request resource in the same order (A $\rightarrow$ B $\rightarrow$ C)

1) Step 1: Lock A

Step 2: Lock B

Step 3: Lock C

Perform transaction

Release C

Release B

Release A

3) Mobile OS App Switching Lag:

1) • Many background apps are in the waiting (I/O) state rather than using the CPU.

• The Scheduler spends time switching between many apps..

7 No, the 20 ms time quantum is hurting performance.

→ With 20 background apps, each app waits longer for CPU time.

→ foreground apps (such as WhatsApp) making app

3> use a smaller Round Robin time quantum (10ms) or a priority scheduling approach where foreground apps receive higher priority.

4> • Faster CPU access for active apps
• Reduced waiting time during app switching

4> Deadlock Risk is on OS:-

1> Yes, this sequence can lead to a deadlock

2> M₁ holds R₁ (Disk I/O) and waits for R₂

3> M₂ holds R₂ (Remot I/O) and waits for R₃

4> M₃ holds R₃ and waits for R₁ (Disk I/O)

M₁ → R₂ → M₂ → R₃ → R₁ → M₁

Banker's Algorithm

• Before allocating a resource, the OS checks whether the system will remain in a safe state

• A resource is allocated only if all modules can

still complete their execution.

- High Priority: M1 -> copy update files
- Medium Priority: M2 -> install drivers
- Medium Priority: M3 -> update registry
- Low: M4 -> validate system integrity

5) Cloud server CPU utilization Analysis :-

$$P = 0.8 \text{ (lower probability)}$$

$$n = 7 \text{ (number of VMs)}$$

$$\text{Formula } \Rightarrow U = 1 - P^n = 1 - (0.8)^7$$

$$U = 1 - 0.2097$$

$$= 0.79$$

$$= 79\%$$

2) CPU utilization = 79%. The server is optimally loaded because the CPU is busy most of the time.

3) effect of changing number of VMs

- High CPU utilization throughout
- Lower throughput and CPU utilization.

4) -> use efficient CPU scheduling

-> reduce I/O waiting using asynchronous I/O

6) Hospital Emergency System Process Scheduling

1) Scheduling issue

Starvation is occurring because emergency request (PI) continuously get CPU time.

suitable scheduling algorithm

Priority Scheduling, with Aging

- 3> use Aging
 - increase the priority of waiting process over time
- 4> Faster response for emergency patients
 - fair CPU allocation.
 - prevents indefinite waiting.
 - Improves overall.

7) E-commerce ware house synchronization

1> synchronization Problem

Race condition.

- 2> Why critical section is required
 - only one thread should update the shared memory
 - at a time prevent incorrect stock count.

3> Recommended solution !

Semaphore (mutex)

4> How it maintain Consistency

- ⇒ Allows only one thread to access the inventory databases
- ⇒ prevents simultaneous updates

8) Video - streaming service!!

1) creating one thread per client cause excessive CPU and memory usage

2) Suitable multithreading model

Thread Pool model

3) Two optimization

- use a thread pool instead of creating
- use asynchronous / non-blocking I/O for client communication

4) $\Rightarrow$ Lower memory usage

$\Rightarrow$ Reduced thread creation overhead.

9) Smart traffic control Deadlock

1) Does Deadlock Exist:

yes

2) Four necessary condition

$T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_4 \rightarrow T_1$

Mutual Exclusion: only one vehicle / process

uses an intersection.

3) Terminate one process (Recovery Technique)

4) $P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4$

This prevents circular wait.

10) University Examination Server Resource

1) Banker's Algo

Before granting a resource request, the system checks whether.

2) Information Required

- available resource
- maximum resource
- remaining need

3) Safe State Check

- Allocate resource temporarily
- Check if all student's sessions

4) Limitation

Banker's algorithm is slow and difficult to use in large cloud system because resource